MODULE 31 · SECTION 4 - PREPARING AND USING DATA FOR ANALYSIS

Embeddings, RAG and LLM-Assisted SQL

Parsing and chunking documents, embeddings, vector indexes and VECTOR_SEARCH, retrieval-augmented generation in BigQuery, Gemini in BigQuery, and reviewing generated SQL.

SECTION 4 of 5 EXAM WEIGHT 15% MODULE 31 of 45

admin@sthithapragna.com

A language model answers well only from the right pages. **Finding those pages is a data job.**

Retrieval-augmented generation in BigQuery

01
Object table
The PDFs stay in Cloud Storage, from module 25.

02
Parse and chunk
Document AI's layout parser, via ML.PROCESS_DOCUMENT.

03
Embed
AI.GENERATE_EMBEDDING turns each chunk into a vector.

04
Retrieve
VECTOR_SEARCH finds the closest chunks.

05
Generate
AI.GENERATE answers, using only those chunks.

THE PAYOFF

Every step is SQL, and the files never leave Cloud Storage to be read.

Cutting documents into useful pieces

- The **layout parser** understands headings, lists and tables, not just raw text.
- Each **chunk** should hold one idea, small enough to match a question precisely.
- Keep the **headings above** each chunk, so 'step 3' still says which procedure.
- Store the source **file and page** with every chunk, so answers can cite them.

> THE TRAP
>
> One vector for a whole 40-page manual matches everything a little, and nothing well.

Meaning as a list of numbers

- An **embedding** places text in a space where similar meanings sit close together.
- **AI.GENERATE_EMBEDDING** embeds a whole table. **AI.EMBED** embeds one value.
- Both call an embedding model through a **remote model** and a connection.
- Embed questions with the **same model** as the documents, or distances mean nothing.

> CURRENCY CHECK
>
> Older material says ML.GENERATE_EMBEDDING. Current docs lead with the AI functions.

Embedding and indexing the manuals

```
1  CREATE OR REPLACE MODEL rag.embedder
2    REMOTE WITH CONNECTION `europe-west2.lk-ai-conn`
3    OPTIONS (endpoint = 'text-embedding-005');
4
5  CREATE OR REPLACE TABLE rag.manual_chunks_emb AS
6  SELECT * FROM AI.GENERATE_EMBEDDING(MODEL rag.embedder,
7    (SELECT chunk_id, uri, content FROM rag.manual_chunks));
8
9  CREATE VECTOR INDEX manual_idx ON rag.manual_chunks_emb(embedding)
10   STORING (uri)
11   OPTIONS (index_type = 'IVF', distance_type = 'COSINE');
```

6-7 Embeds every chunk. The text column is named content.

9-10 An IVF index on the vectors, storing the file for filters.

HONEST LIMIT

Endpoint names change as models are replaced. Check the current model list before you build.

Embed in batches, or autonomously?

AI.GENERATE_EMBEDDING

You run it

- A query builds an embedding table
- You rerun it when sources change
- Search with VECTOR_SEARCH
- Easy to forget a refresh

AUTONOMOUS GENERATION

BigQuery keeps it in sync

- A generated column, using AI.EMBED
- Inserts and updates are embedded for you
- Search simply with AI.SEARCH
- One such column per table

THE TIEBREAKER

Sources change often, and nobody should have to remember: autonomous embedding generation.

Exact or approximate?

- Without an index, **VECTOR_SEARCH** checks every row: exact, but slow on big tables.
- **CREATE VECTOR INDEX** gives approximate search: faster and cheaper, recall below 100%.
- **IVF** for small batches of queries. **TreeAH** for hundreds at once, up to 200 million rows.
- Tables under **10 MB** are not indexed, and Standard edition cannot use indexes.

REMEMBER

New rows are searched exactly until the index catches up, so nothing is missed.

Filters, recall and monitoring

- **STORING** columns let a filter run before the search, such as one meter model.
- Filters on other columns run afterwards, and can return fewer than **top_k**.
- Raise **top_k**, or **fraction_lists_to_search**, to win back recall.
- **INFORMATION_SCHEMA.VECTOR_INDEXES** shows coverage and status.

> THE TRAP
>
> Stored columns are ignored on tables with row-level security. Plan filters around that.

Answering an engineer's question

```
1  WITH hits AS (
2    SELECT base.uri, base.content
3    FROM VECTOR_SEARCH(TABLE rag.manual_chunks_emb, 'embedding',
4      (SELECT embedding FROM AI.GENERATE_EMBEDDING(MODEL rag.embedder,
5        (SELECT 'How do I reset a meter after error E42?' AS content))),
6      top_k => 5, distance_type => 'COSINE'))
7  SELECT AI.GENERATE(
8    CONCAT('Answer only from these extracts, and cite the file. ',
9           STRING_AGG(CONCAT(uri, ': ', content), ' | ')),
10   connection_id => 'europe-west2.lk-ai-conn').result AS answer
11 FROM hits;
```

3-6 Embed the question with the same model, then take the five closest chunks.

8 Ground the answer in the extracts, with citations.

THE PATTERN

If the answer is wrong, look at the five chunks first. Retrieval fails more often than generation.

Gemini as a SQL assistant

- **SQL generation** from plain language, completion as you type, and explanations.
- **Data canvas**: find tables, join, query and chart, all in natural language.
- **Data insights** suggests queries from table metadata. **Data preparation** suggests fixes.
- The **Data Engineering Agent** builds and troubleshoots pipelines from prompts.

> FOR LARKSPUR
>
> Good descriptions and glossary terms, from module 26, make every one of these work better.

Plausible is not the same as right

- Google warns that output can seem **plausible but be factually incorrect**.
- Read the **joins and filters**. Check the grain, the dates and the partition filter.
- **Dry run** it for bytes, then compare with a figure you already trust.
- Gemini does not train on your data, but lacks some BigQuery **compliance** offerings.

> THE TRAP
>
> Asking Gemini to explain its own query is not a review. A person must check it.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Answers **vague**, whole documents embedded	Layout parser, chunk with headings
Semantic search, **too slow** at scale	Vector index, accept approximate recall
Embeddings go **stale** as sources change	Autonomous embedding generation
Search by **meaning**, not keywords	VECTOR_SEARCH or AI.SEARCH
Exact keyword or ID **lookup**	Search index, from module 28
Generated SQL looks **plausible**	Review, dry run, compare to a known figure

THE TIEBREAKER

Meaning needs vectors. Exact strings need a search index. Do not mix them up.

> The field engineer assistant gives vague answers about meter procedures.
>
> The pipeline stores one embedding for each manual, and some manuals run to 40 pages.

What should the team change first?

A Switch to a larger embedding model with more dimensions

B Raise top_k to 50, so that more manuals reach the prompt

C Parse with the layout parser, and embed each headed chunk

D Lower the generation model's temperature, for steadier answers

◆ PAUSE AND DECIDE

What should the team change first?

A Larger embedding model

Still one vector per forty pages.

B top_k of 50

Fifty whole manuals in the prompt, still unfocused.

C Chunk, then embed

Each vector holds one idea, so matches become precise.

D Lower temperature

Steadier wording, built on the same wrong extracts.

THE SIGNAL

Vague answers from whole-document vectors: chunk first.

> Engineers' questions search 30 million chunk embeddings, one question at a time.
>
> Each search scans every row, and is too slow and too costly. Slightly approximate results are acceptable.

What should the team add?

A An IVF vector index on the embedding column

B A search index on the chunk text column, for SEARCH

C Clustering of the table on the embedding column

D AI.SIMILARITY comparisons against every row in the table

◆ PAUSE AND DECIDE

What should the team add?

A IVF vector index

Approximate nearest neighbours, fast for single queries.

B Search index

Exact keyword lookups, not meaning.

C Cluster on embeddings

Arrays of numbers cannot be cluster keys.

D AI.SIMILARITY everywhere

Meant for a few comparisons, not millions.

THE SIGNAL

Semantic search too slow, approximate is fine: a vector index.

> Engineering updates manuals every week, and someone is meant to rerun the embedding query afterwards.
>
> They often forget, so new procedures never show up in search results.

What should the team do?

A Add a weekly calendar reminder to rerun the embedding query

B A generated embedding column using AI.EMBED, kept in sync

C Rebuild the vector index every day, on a scheduled query

D Raise fraction_lists_to_search, so the index searches wider

◆ PAUSE AND DECIDE

What should the team do?

A Calendar reminder

Still depends on someone remembering.

B Autonomous embeddings

New and changed chunks are embedded automatically.

C Rebuild the index

Indexes vectors that were never regenerated.

D Search wider

Cannot find chunks that were never embedded.

THE SIGNAL

Embeddings that must track changing sources: autonomous generation.

> An analyst used Gemini in BigQuery to write a quarterly revenue query for the board pack.
>
> The figures look plausible. The board pack goes out tomorrow.

What should happen before the figures are used?

A Publish them, since Google checks the SQL that Gemini writes

B Ask Gemini to explain the query, and accept its explanation

C Turn off the query cache, so the figures are fully fresh

D Check joins and filters, dry run, compare to a known total

◆ PAUSE AND DECIDE

What should happen before the figures are used?

A **Publish now**
Google warns output can be plausible but wrong.

B **Gemini explains itself**
A model marking its own work is not a review.

C **Cache off**
Fresher numbers, the same possible logic error.

D **Human review**
Checks logic, cost, and the answer itself.

THE SIGNAL

Generated SQL for decisions: a person reviews it and checks the result.

Module 31 in five lines

- RAG in SQL: object table, parse and **chunk**, embed, search, generate.
- Chunks keep their **headings**. Questions use the **same** embedding model.
- **Vector indexes** trade a little recall for speed. IVF for single queries.
- **Autonomous embeddings** keep vectors in step with changing sources.
- Gemini writes SQL fast. A **person** still reviews it before it matters.

COMING UP

Next: module 32, sharing data with BigQuery sharing and clean rooms.